

Connect IQ App — Dual-Tank Anaerobic Reserve Tracker

Spec + ready-to-use build prompt for a Garmin Connect IQ data field that tracks the reserve, consumption, and depletion of the PCr (alactic) and glycolytic (lactic) systems in real time from power.

Companion to `white-paper-dual-tank-anaerobic-model.md` (the model this app implements).

Part 1 — Why these platform choices (grounding for the prompt)

Before the prompt, the design decisions it encodes, so you can adjust them:

- **App type = custom Data Field**, not a widget or full app. A `Toybox.WatchUi.DataField` runs *inside* the native activity recording, so it gets live sensor data, records to the FIT file automatically, and its `compute(info)` is invoked **once per second** — which is exactly the model's $\Delta t = 1\text{ s}$ integration step. No custom timer needed.
- **Power source = `Activity.Info.currentPower`** (Watts, from a paired power meter). Guard for `null` (coasting, sensor dropout) by treating it as 0 W.
- **Custom rendering** via `onUpdate(dc)` because we draw two gauges, not a single number. This means it must be a *full-screen* / custom layout data field, which limits it to one field slot but gives full drawing control.
- **Settings via `Application.Properties` + `settings.xml` / `properties.xml`** so CP , W' , and the tank constants are entered in Garmin Connect / Connect IQ app store settings, not hard-coded.
- **FIT recording via `FitContributor`** — expose PCr%, glycolytic%, and consumption as recorded fields so they land in Garmin Connect and sync onward to `intervals.icu` / Strava for post-ride review.
- **State = two `Float`s** (rP , rG) held on the field instance. Persist to `Storage` in `onTimerLap` / `onTimerStop` optionally; a fresh ride starts full.
- **Language = Monkey C**, min SDK targeting devices with power + Connect IQ 3.x+ (Edge 530/540/830/840/1030/1040, Forerunner 255/955/965, etc.).

Memory/CPU: the whole model is ~ 10 float ops + 2 `Math.exp` per second — negligible. Keep the draw code allocation-free (no `new` inside `onUpdate`) to stay within data-field memory budgets.

Part 2 — The model to implement (reduced dual-tank, from the white paper §4)

State: rP (PCr reserve, J), rG (glycolytic reserve, J). Params (from settings): CP , W_{prime} , fP , pP_{max} , τ_P , τ_G , lt1Frac , η . Derived: $cP = fP \cdot W_{\text{prime}}$, $cG = (1 - fP) \cdot W_{\text{prime}}$. Init $rP = cP$, $rG = cG$. Session accumulators (J, ride totals): $\text{dep}P = 0$, $\text{dep}G = 0$ — total energy ever drawn from each tank.

Each second, with power P and $dt = 1$:

```

delta = P - CP
if delta > 0:                                # DEPLETION (above CP)
    need  = delta * dt
    takeP = min(need, rP, pPmax*dt)         # PCr first, rate-limited
    rP    -= takeP; need -= takeP
    takeG = min(need, rG)                   # glycolytic covers remainder
    rG    -= takeG; need -= takeG
    depP += takeP; depG += takeG            # accumulate lifetime depletion (J)
    exhausted = (need > 0)
    consP = takeP/dt; consG = takeG/dt      # live consumption (W)
else:                                        # RESTORATION (at/below CP)
    rP += eta * (cP - rP) * (1 - exp(-dt/tauP))
    if P < lt1Frac*CP:
        gate = (lt1Frac*CP - P) / (lt1Frac*CP)
        rG += gate * (cG - rG) * (1 - exp(-dt/tauG))
    consP = 0; consG = 0

clamp rP to [0,cP], rG to [0,cG]
pctP = 100*rP/cP; pctG = 100*rG/cG; pctW = 100*(rP+rG)/Wprime
kJ_P = depP/1000; kJ_G = depG/1000          # session totals depleted per system (kJ)

```

Optional (behind a setting): fatigue-slowed PCr recovery $\tau_{Peff} = \tau_P * (1 + k * (1 - rG/cG))$, and an aerobic-ramp so the CP boundary is a first-order $A(t)$ toward $\min(P, CP)$ with $\tau_{Aer} \approx 25$.

Part 3 — THE BUILD PROMPT

Copy everything in this block into your coding agent (or use it yourself). It is written to produce a complete, compilable Connect IQ project.

You are building a Garmin Connect IQ **data field** in Monkey C that tracks two anaerobic energy reserves – phosphocreatine (PCr/alactic) and glycolytic (lactic) – live from cycling power. Produce a complete, buildable project (manifest, source, resources, settings).

Deliverables

1. `manifest.xml` – a datafield app; permissions for Sensor/FitContributor; product list covering Edge 530/540/830/840/1030/1040 and Forerunner 255/955/965; min SDK 4.0.
2. `source/DualTankView.mc` – a class extending `Toybox.WatchUi.DataField`.
3. `source/DualTankApp.mc` – `Toybox.Application.AppBase` returning the data field.
4. `resources/settings/settings.xml` + `resources/settings/properties.xml` – user config.
5. `resources/strings/strings.xml`, `resources/layouts/` if needed.
6. `monkey.jungle` build file. Plus a short README with build/sideload steps.

Model (implement EXACTLY this; dt = 1 second = one compute() call)

State (Float, held on the view): rP (PCr reserve, J), rG (glycolytic reserve, J), depP (total J ever drawn from PCr this session), depG (total J ever drawn from glycolytic).

Settings (with defaults): CP=250 (W), Wprime=20000 (J), fP=0.35, pPmax=300 (W), tauP=22 (s), tauG=360 (s), lt1Frac=0.80, eta=0.80.

Derived: cP=fP*Wprime, cG=(1-fP)*Wprime. On init and onTimerStart-from-fresh: rP=cP, rG=cG, depP=0, depG=0.

Each compute(info):

```
P = info.currentPower has value ? info.currentPower : 0
delta = P - CP
if delta > 0:
    need = delta
    takeP = min(need, rP, pPmax); rP -= takeP; need -= takeP
    takeG = min(need, rG); rG -= takeG; need -= takeG
    depP += takeP; depG += takeG      # session-total depletion accumulators (J)
    exhausted = need > 0; consP = takeP; consG = takeG
else:
    rP += eta*(cP - rP)*(1 - Math.exp(-1.0/tauP))
    if P < lt1Frac*CP:
        gate = (lt1Frac*CP - P)/(lt1Frac*CP)
        rG += gate*(cG - rG)*(1 - Math.exp(-1.0/tauG))
    consP = 0; consG = 0
clamp rP in [0,cP], rG in [0,cG]
pctP = 100*rP/cP; pctG = 100*rG/cG
```

Guard against CP<=0 and Wprime<=0 (skip update, show "SET CP/W").

Rendering (onUpdate(dc)) – ASPECT-ADAPTIVE TWO-BAR LAYOUT

```

Pick orientation from the field's aspect ratio (`w`, `h` from
`dc.getWidth()/getHeight()`):
- Wide/short slot (`w*2 >= h*3`): two horizontal bars STACKED (top = PCr, bottom =
GLY),
    each filling LEFT→RIGHT; label on the left, "%" on the right.
- Square or tall slot (e.g. a 1×2 cell): two vertical bars SIDE BY SIDE, each
filling
    BOTTOM→TOP; label above, "%" below.
Live consumption ("-NNW") is drawn inside the bar while that system drains. Same color
rules in
both orientations.

Color logic (per gauge, decided each frame):
- Idle / recovering (not being drained this frame): DULL, desaturated fill.
- Actively depleting (consumption > 0 this frame): BRIGHT, saturated fill.
- Depleted (reserve at/near 0, i.e. exhausted or pct ≤ ~3%): RED, and FLASH.
Base hues are fixed per system – PCr = purple, GLY = green:

| System | Idle (dull) | Depleting (bright) | Depleted |
|---|---|---|---|
| PCr (purple) | `0x5A3A6E` muted purple | `0xB44DFF` bright purple | `0xFF0000` flashing
|
| GLY (green) | `0x2E5A3A` muted green | `0x37E85A` bright green | `0xFF0000` flashing
|

- "Depleting this frame" = `consP > 0` (PCr bar) / `consG > 0` (GLY bar). Track these as
state set in compute().
- Flash = toggle the depleted bar between red and background each ~0.5 s. Since
compute()/onUpdate
    run at 1 Hz, keep a boolean `flashOn` that flips every update and use it only when that
bar is depleted;
    request extra redraws with `WatchUi.requestUpdate()` if you want a faster blink than 1
Hz.
- Draw an empty-track outline (thin) behind each bar so 0% is still visible; fill = the
color above.
- Overlay per bar: left-aligned label ("PCr"/"GLY"), right-aligned "NN%", and the live
draw
    ("-180W") shown only while that bar is depleting.
- Optional thin combined-W'bal tick/number (pctW) in a corner; keep it small.
- Optional (if space): show running session totals "PCr N.n kJ / GLY N.n kJ" (depP/1000,
depG/1000)
    as small text – the same numbers written to the session FIT fields.
- Respect getObscurityFlags()/full-screen vs partial layouts; use
dc.getWidth()/getHeight();
    precompute fonts/colors in onLayout, no allocation inside onUpdate.
- Handle dark/light device themes via getBackgroundColor() (the dull hues above read on a

```

dark

background; if background is white, darken the dull fills ~15% for contrast).

FIT recording (FitContributor) – REQUIRED

Create fields in initialize() via createField(...). Two kinds:

RECORD-level (written EVERY compute() = once per second, so they become 1 Hz streams in the FIT):

- "PCr_pct" FLOAT, units "%", MSG_TYPE_RECORD → PCr reserve level each second
- "GLY_pct" FLOAT, units "%", MSG_TYPE_RECORD → glycolytic reserve level each second
- "PCr_cons" SINT16, units "W", MSG_TYPE_RECORD → live PCr draw (optional but useful)
- "GLY_cons" SINT16, units "W", MSG_TYPE_RECORD → live glycolytic draw (optional)

You MUST call setData() on the two *_pct fields on every compute() tick (even during recovery

and when paused-but-recording is false, guard so you only write while the activity is recording),

guaranteeing a per-second reserve trace for the whole ride.

SESSION-level (single summary value finalized at ride end):

- "PCr_depleted_kJ" FLOAT, units "kJ", MSG_TYPE_SESSION → total energy drawn from PCr = depP/1000

- "GLY_depleted_kJ" FLOAT, units "kJ", MSG_TYPE_SESSION → total energy drawn from glycolytic = depG/1000

Update these each compute() with the running depP/1000 and depG/1000; the FIT SDK keeps the last

value as the session summary, so the recorded totals reflect the whole session at save time.

Optionally also add session mins: "PCr_min_pct", "GLY_min_pct".

Field IDs must be unique small integers. These record-level streams and session totals sync to

Garmin Connect and flow on to intervals.icu / Strava as custom data.

Settings (settings.xml / properties.xml)

Expose CP, Wprime, fP, pPmax, tauP, tauG, lt1Frac, eta as editable properties with the defaults

above, sensible min/max, and clear titles/descriptions. Read them in onSettingsChanged() (or on

each compute if simpler) via Application.Properties.getValue, recomputing cP/cG and re-clamping.

Lifecycle

- compute(info) does the model step and returns a value (e.g. pctP) for single-field fallback.

- onTimerStart / onTimerReset: initialize reserves to full (rP=cP, rG=cG) AND zero the session depletion accumulators (depP=0, depG=0) if starting fresh.

```

- onTimerPause/onTimerStop: freeze depletion (no accumulation while paused) and stamp the
  pause
  time. onTimerResume (or a start from a stopped-not-reset timer): recover both tanks in
  closed
  form for the ENTIRE elapsed pause at rest — `cap - (cap-R)*(1-a)^N` for N pause-seconds
- so a
  long stop refills correctly even if the device stops calling compute() while paused.
- Be null-safe on all Activity.Info fields.
- Realism terms (implemented, tunable): aerobic ramp (`tauAer`, first-order aerobic supply
  toward
  min(P,CP); 0 = hard CP) and fatigue-slowed PCr recovery (`tauPeff = tauP*(1+fatK*(1-
  rG/cG))`;
  fatK=0 disables). See the scaffold in `../connectiq/`.

## Quality bar
- Compiles with the Connect IQ SDK; no runtime allocation in onUpdate; documented
  constants;
  a UNITS/ASSUMPTIONS comment block noting energies are in Joules, dt=1s, and that fP is a
  modeling choice. Include 3–4 inline test-trace comments describing expected behavior
  (single sprint drains PCr and recovers in ~30–60s; sustained supra-CP drains GLY which
  recovers over minutes only below LT1).

```

Part 4 — UI sketch (what the prompt should produce)

Two stacked **horizontal** bars, filling left → right. Color = system hue, brightness = whether that tank is being drained right now.

```

| PCr ██████████░░░░ 78% | ← dull purple: not draining
(recovering/steady)
|
| GLY ██████████░░░░ 41% -180W | ← bright green: draining now, shows live
watts
|                                     W' 59%|
|_____|

depleted state (reserve ~0):
| GLY ██████████ 0% ▲ | ← solid RED, flashing on/off ~2 Hz

```

- **Brightness encodes action:** a bar sitting still (below-zone, steady, or recovering) is the **dull** hue; the instant it starts supplying energy (that system's consumption > 0) it snaps to the **bright** hue — so at a glance you see *which* tank the surge is coming out of.

- **PCr bar (purple)** = "how much punch is left" — moves fast, flicks bright on any hard jump, refills (dull) in seconds of soft-pedaling.
- **GLY bar (green)** = "how much sustained dig is left" — moves slower, goes bright on sustained supra-CP efforts, only refills when you drop below LT1, and comes back over minutes.
- **Either bar** → **solid red + flash** when its tank empties, the "you've spent this system" warning.
- Flash / red border when either tank hits ~0 (`exhausted`).

Part 5 — Test plan (hand these traces to the built app)

Trace	Expected
5 s @ 800 W, then 60 s @ 100 W (CP=250)	PCr drops sharply, GLY barely moves; PCr ~fully back by ~45–60 s
3 min @ 300 W (supra-CP hold)	PCr empties in the first ~20–40 s, then GLY bleeds steadily; both low at end
8×(20 s @ 400 W / 40 s @ 120 W)	PCr sawtooths (drains/refills each rep); GLY trends down across the set
20 min @ 150 W (endurance)	both stay ~full; GLY slowly tops off since $P < LT1$
sum(PCr+GLY) vs reference W'bal	matches a Froncioni–Clarke W'bal integrator within tolerance

Part 6 — Beyond the head unit (optional extensions)

- **intervals.icu / Strava:** because reserves are recorded as FIT fields, they appear as custom streams post-ride — no extra work. A companion server model could re-fit `fP` / `τ` from each ride.
 - **Live alerts:** trigger a Connect IQ tone/vibe when $PCr < 20\%$ (don't attack) or $GLY < 20\%$ (can't sustain).
 - **Per-athlete calibration:** a one-time "sprint then hold" test to estimate `pPmax`, `fP`, and `τ_p` instead of defaults (see white paper §6).
 - **Alternate targets:** the same model + prompt structure ports to Wahoo (no Connect IQ; would need their SDK) or a phone app reading BLE power.
-

This app implements the reduced dual-tank model in `white-paper-dual-tank-anaerobic-model.md`. Tank levels are model estimates for pacing, not measurements of muscle chemistry; calibrate and validate (white paper §6–7) before trusting absolute numbers.